Computer Methods and Programs in Biomedicine 273 (2026) 109123

Contents lists available at ScienceDirect

Computer Methods and Programs in Biomedicine

journal homepage: https://www.sciencedirect.com/journal/computer-methods-and-programs-in-biomedicine

A real-time reconstructed grid method for soft tissue cutting and haptic

Liang Li, Lai Zhou *,1, Xueyu Zhou

Pre-research Department, China Simulation Sciences Co. Ltd., No. 1 Kangqiao East Road, Building 2, Pudong New District, Shanghai, 201315, China

ARTICLE INFO

Keywords:
Virtual surgery
Soft tissue cutting
Haptic

ABSTRACT

Background and Objective: The virtual surgery system is crucial for medical training and preoperative planning. To address the issues faced in existing research, such as distorted grids, cutting with volume loss, low computational efficiency, and surgical tool penetration, this paper proposes a parallel data structure optimized for GPU computation. This structure enables efficient cutting with minimal volume loss and provides force feedback effects without model penetration.

Methods: This paper applies the Dual Contouring algorithm within the deformable grid framework for real-time mesh reconstruction, which can be divided into surface reconstruction during deformation and cutting surface reconstruction. By precomputing all possible voxel grid connections and the calculation methods for the feature points of each mesh, feature point positions can be quickly determined within a single GPU thread, accelerating the reconstruction process. Additionally, the force feedback algorithm based on Signed Distance Field and the Finger-proxy model addresses the penetration issue when the surgical tool collides with soft tissue.

Results: Simulation results show that, compared to traditional mesh cutting methods, the proposed approach retains the non-manifold surface features at the cut areas during real-time cutting, and the frame rate does not decrease as the number of cuts increases. The use of the Signed Distance Field for collision detection ensures that the tool has both collision volume and force feedback effects.

Conclusion: The proposed method can handle interactive cutting and collision operations in various surgical simulations and achieves an interactive frame rate that meets real-time requirements, contributing to more efficient and realistic virtual surgery simulations.

1. Introduction

With the continuous evolution of computer hardware and software technologies, alongside iterative advancements in force feedback interaction devices, virtual surgery systems are playing an increasingly vital role in the medical field. In virtual environments, three-dimensional geometric and physical modeling of human organ models, combined with haptic algorithms and devices for simulated surgical operations [1], enables specific organs or tissues to be repeatedly utilized. These approaches hold significant importance for preoperative planning, training, reducing the demand for real samples, and mitigating the risk of disease transmission [2].

Interactive simulation of soft tissues primarily involves research into key technologies such as physical modeling, geometric modeling, and force-tactile feedback. The nonlinear, anisotropic mechanical behavior of biological soft tissues [3] is significantly more complex than that of isotropic soft tissues, making deformation simulation a prerequisite for visual and force-tactile feedback in simulation systems. The mechanical modeling of biological soft tissues is typically achieved by establishing physically based constitutive equations. This process involves dividing the soft tissue model into smaller mechanical units and adjusting their poses based on local connectivity relationships to control the overall shape of the model [4]. Commonly used modeling methods include the Mass-Spring Model (MSM), Finite Element Method (FEM), and Position-Based Dynamics (PBD) [5–7]. Beyond addressing the complexities of nonlinear biomechanical models, real-time modifications to the three-dimensional topological structure of soft tissues, including procedures such as cutting, electrocoagulation, and tearing, represent a critical area of focus. Precisely replicating the deformation behavior of soft tissue organs during interactive processes poses a major challenge in contemporary research on virtual surgery technology.

In recent years, researchers have made significant progress in exploring various physical modeling techniques for soft tissues. When employing mesh-based methods, detailed spatial descriptions of mesh segmentation enable precise representation of complex boundaries and

* Corresponding author.
E-mail addresses: liang.li@csssim.com (L. Li), lai.zhou@csssim.com (L. Zhou), xueyu.zhou@csssim.com (X. Zhou).
[1] These authors contributed equally to this work.

https://doi.org/10.1016/j.cmpb.2025.109123
Received 20 March 2025; Received in revised form 24 September 2025; Accepted 15 October 2025
Available online 20 October 2025

internal structures [8,9]. In finite element simulations, achieving high-precision soft tissue modeling requires high-quality mesh division and numerical integration, which demands substantial computational resources. Consequently, the Finite Element Method (FEM) is better suited for offline simulations [10]. To enhance computational performance and enable FEM for real-time soft tissue deformation simulation, researchers have pursued further advancements. Marinković et al. proposed a simplified geometrically nonlinear co-rotational finite element formulation, enhanced by coupled mesh techniques, allowing complex geometric shapes to be modeled with relatively simple computational models [11]. For specific soft tissue organs, the Corotational Cut Finite Element Method can be employed, utilizing background meshes and multilayer embedding algorithms to automatically capture complex geometries. Surface details are derived from binarized images, bypassing the complexity of mesh generation and enabling real-time needle insertion simulations for various soft tissues without requiring mesh conformity [12]. Additionally, by discretizing FEM and converting the effect of external forces on soft tissue deformation into a constraint estimation problem, deformation behavior can be estimated from local surface displacement data, thereby improving FEM computational efficiency [13]. Compared to FEM, the Mass-Spring Model (MSM) is frequently used in real-time simulations due to its lower overall computational cost and high adaptability to geometric topological structures [14]. However, when simulating high-stiffness deformations, large time steps can compromise stability, and parameter tuning remains challenging. To enhance its accuracy and stability, researchers have introduced a series of improvements and applied them to various soft tissue modeling scenarios. Duan et al. utilized a predict–correct method to refine explicit integration results, incorporating nonlinear properties and volume constraints to reflect biomechanical characteristics and the incompressibility of soft tissues in simulations [15]. Beyond integrating MSM into traditional volumetric meshes, modeling solely on surface triangular meshes can also preserve volume. Hongly Va et al. leveraged the divergence theorem and implicit constraints to construct MSM within surface triangular meshes, achieving stable deformation and volume preservation while significantly reducing computational complexity compared to traditional volumetric meshes [16]. To better capture force propagation [17] and modeling precision, Zhang et al. combined convolutional neural networks (CNNs) with traditional physical models, incorporating biomechanical parameters and optimizing the model via neural networks to achieve heterogeneous soft tissue deformation effects [18].

In simulating soft tissue cutting, real-time coupling of biomechanical properties with dynamic meshes is essential, requiring both high-precision physical model representations and adaptive updates to topological structures [19]. Wu et al. performed physical modeling based on tetrahedral mesh structures, implementing liver resection by subdividing and removing tetrahedra at cutting edges. However, the triangles generated on cutting surfaces depend on tetrahedral size, resulting in coarse meshes that fail to accurately represent the incision [20]. Jerábková et al. used refined hexahedral voxels with the Marching Cubes (MC) algorithm for real-time geometric modeling and coarse mesh models for dynamic modeling, achieving cutting by removing refined voxels and updating surfaces. However, repeated cuts led to noticeable volume loss [21]. Fortunately, some volume loss also occurs in real electrocoagulation cutting. Berndt et al. simulated monopolar electrocoagulation cutting of soft tissues using Position-Based Dynamics (PBD) and MC algorithms, dynamically adjusting the positional relationship between hexahedral voxel vertices and isosurfaces to exclude specific vertices during surface reconstruction, thereby achieving electrocoagulation effects [22]. Yu et al. built on this by developing an Aggregated Finding Connected Components (AFCC) algorithm to address discrepancies between visual meshes and simulation models during cluster separation, avoiding ghost forces and visual artifacts in discontinuous cutting [23]. To produce low-loss cutting surfaces, Wu et al. employ a coarse grid for mechanical modeling combined with adaptive grids for geometric modeling, achieving better smoothness and visual quality of the cutting surfaces [24,25]. Chen et al. improved tetrahedral subdivision methods, employing progressive planar cutting while simultaneously performing optimization and simulation to ensure consistency between topological structures and force models, enabling more flexible cutting paths. However, such mesh-based cutting approaches inevitably generate a large number of simulation elements after multiple cuts, leading to reduced solving speeds [26]. In addition to traditional mesh-based cutting methods, volumetric models with both surface and internal structures can be constructed from CT/MRI data. By further optimizing the incision shape using second-order Bézier curves, it becomes possible to achieve partial cuts that preserve internal features [27,28].

Although current technical approaches can meet specific requirements for soft tissue modeling or cutting under constrained conditions, simulations often focus on individual functionalities in isolation. From a visual standpoint, achieving low-volume-loss cutting while maintaining a smooth frame rate and user experience remains challenging. Regarding force feedback, traditional methods typically rely on applying increased reaction forces to prevent penetration. While these approaches can be effective within moderate force ranges, excessive user input may still lead to undesirable outcomes—such as excessive compression of the tissue or visual artifacts like interpenetration.

This paper proposes a CUDA-parallelized Dual Contouring algorithm for real-time three-dimensional model reconstruction. The proposed method generates low-loss cutting surfaces during progressive soft tissue cutting simulations and prevents the physical simulation of soft tissues from being adversely affected by elongated triangular facets after multiple cuts. The paper employs structural springs and bending springs for soft tissue physical modeling [29], utilizes an adaptive Runge–Kutta method for updating particle positions [30], and leverages GPU acceleration for particle position calculations, ensuring both the accuracy and real-time performance of soft tissue modeling. To tackle the issue of penetration between soft tissues and surgical instruments during interaction, this paper introduces a method combining Signed Distance Functions with the Finger-Proxy model to determine the optimal position of surgical instruments following collisions with soft tissues. This method ensures that, upon collision, the surgical instrument exerts appropriate pressure and friction forces on the soft tissue, preventing protrusions or incorrect displacements of particles within the collision region due to penetration. Furthermore, the paper designs a soft tissue interaction system accommodating multiple operating conditions, achieving smooth progressive cutting and stable force feedback effects. The primary contributions of this paper are summarized as follows:

- Propose a parallel-accelerated Dual Contouring algorithm to achieve real-time three-dimensional reconstruction under both fine cutting and electrocoagulation cutting processes, while integrating a nonlinear physical modeling approach to ensure that both visual and physical effects are accurately represented simultaneously.
- Propose a force feedback algorithm based on signed distance fields, effectively preventing the loss of force feedback or incorrect deformation of soft tissues caused by model penetration during collisions between surgical instruments and soft tissues.
- Design cutting and collision simulation experiments to validate the proposed method, with results demonstrating that the method can generate smooth and realistic incisions while adapting to various operating conditions and collision interactions.

Rest of paper is organized as follows: Section 2 introduces the cutting, soft tissue modeling, and collision interaction methods. Section 3 provides a detailed description of the virtual simulation platform and multi-threaded computation framework. Section 4 presents the simulation process. Conclusions are drawn after simulation results.

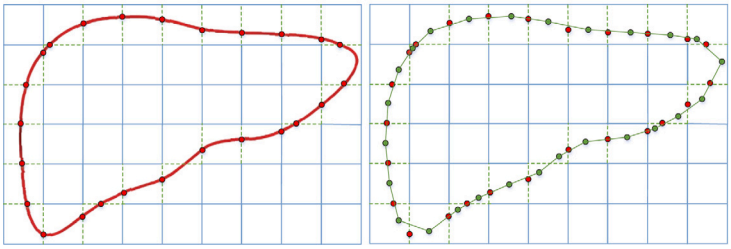

Fig. 2.1. Process of geometric surface rendering for soft tissues.

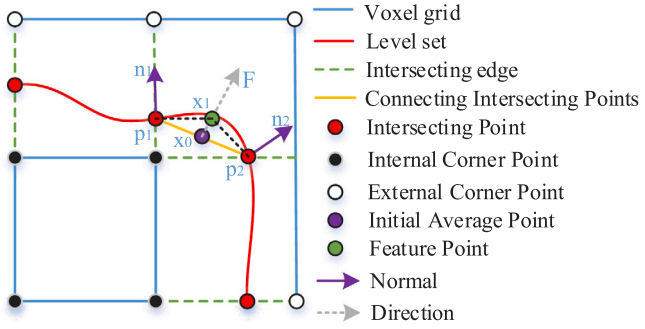

Fig. 2.2. Feature point update method.

2. Methods

To preserve edge features during real-time cutting and ensure that multiple cuts do not degrade frame rates, this paper adopts the Dual Contouring (DC) method for surface reconstruction. As illustrated in Fig. 2.1, the input volumetric data is divided into a uniform background grid, followed by adaptive octree subdivision. The intersection properties of each cube with the isosurface are recorded, including the local coordinates of intersection points and the normal vectors of the outer surface. Feature point positions are then computed, and adjacent voxel feature points are connected to form an approximate surface.

$$E = \sum_{i=0}^{n-1} \left(\left(\boldsymbol{n}_i \cdot (\boldsymbol{x} - \boldsymbol{p}_i) \right)^2 \right) \tag{1}$$

The above represents the quadratic error function for feature points, where $\boldsymbol{n}_i$ and $\boldsymbol{p}_i$ denote the intersection point and normal of the ith edge of the cube, respectively, and $\boldsymbol{x}$ is the feature point position. By minimizing the value of E, an approximation of $\boldsymbol{x}$ can be obtained. To avoid performance degradation caused by using matrix decomposition to solve the Quadratic Error Function (QEF), this paper adopts the method proposed in [31]. The approach utilizes a particle movement approximation and Hermite data to generate a single feature point within the cube at the boundary. Furthermore, spatial constraints and gradient information are employed to optimize the vertex positions.

As shown in Fig. 2.2, the initial feature points are obtained by first calculating the average of all intersection points within the grid. The following relationships can be derived:

$$\boldsymbol{x}_0 = \sum_{i=1}^{n} \frac{\boldsymbol{p}_i}{n} \tag{2}$$

The feature point motion direction $\boldsymbol{F}$ is obtained based on the positions of the intersection points and their normals:

$$\boldsymbol{F} = \sum_{i=1}^{n} \left(- \left(\boldsymbol{x}_0 - \boldsymbol{p}_i \right) \cdot \boldsymbol{n}_i^2 \right) \tag{3}$$

The new feature point position is written as following,

$$\begin{cases} \boldsymbol{x}_1 = \boldsymbol{x}_0 + a \cdot \boldsymbol{F} \\ a = 0.1 \cdot \left(1 - \frac{i}{m} \right) \end{cases} \tag{4}$$

where a is used to scale the update step, allowing the feature points to converge to their final positions more quickly. The maximum number of iterations is m. At this point, based on all the edges that intersect with the iso-surface, the feature points of the surrounding four cubes are connected to form the outer surface. This method shares the same limitation as matrix decomposition, where face intersections may occur, primarily because the feature points extend beyond their respective voxels. Let the voxel edge length be L and the position be (x,y,z), then the signed distance field can be obtained:

$$SDF(x, y, z) = \max(\max(|x - x_c| - \frac{L}{2}, |y - y_c| - \frac{L}{2}), |z - z_c| - \frac{L}{2}) \tag{5}$$

when the SDF value is negative, the iteration is stopped, resulting in a smooth, non-intersecting surface.

2.1. Cutting process

2.1.1. The static cutting process

As shown in the Fig. 2.3, the surface reconstruction and cutting diagram in 2D, the deformable object is embedded in the background grid, and its surface is reconstructed using Dual Contouring. The reconstruction of the outer surface is the preprocessing step before the cutting, and its complexity does not affect the real-time performance. In this process, the blue grid represents the voxel grid used for reconstruction, where the 12 edges of each voxel are used both for 3D reconstruction and to represent the physical interactions between particles. When reconstructing the outer surface, the voxel edges that intersect with the isosurface are marked as "Intersecting edges", and the intersection points generated on the voxel edges are marked as "Intersecting Points". The global coordinates of these intersection points, their local coordinates within the voxel, and the normals at these points are also recorded. To quickly identify the intersecting edges, a Marching Cubes lookup table is used. Each voxel intersecting with the isosurface computes the feature points as shown in Fig. 2.2. Then, for each intersecting edge, the feature points from the surrounding four voxels are connected to form two triangular surfaces. The figure on the right illustrates the specific cutting process, where the yellow line represents the cutting path. During the cutting process, the voxel edges being cut are marked as "Cutting edge", and the intersection point location information is recorded. The surface generation during the cutting process differs from that of the outer surface. The cut internal voxel grid will dynamically compute its undirected graph connectivity. This graph has 8 vertices and 12 edges, and each voxel, when cut, can form up to 8 feature points. Therefore, the connectivity component of each vertex needs to be obtained. The connectivity of the voxel is pre-calculated and stored as a 2D array, with a total of 2^{12}(4096) possible configurations. Similar to the Marching Cubes lookup table, each edge has two states: intersecting with the target contour or not. Thus, the state of each edge can be represented by a binary bit, where

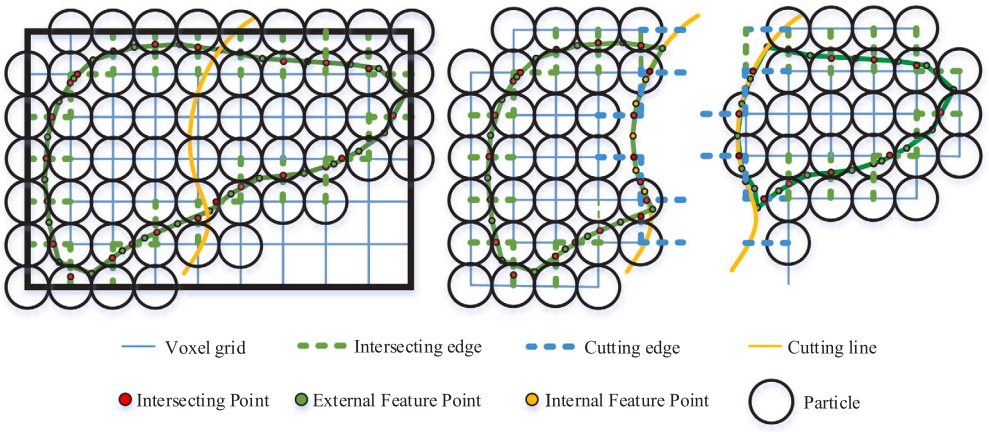

Fig. 2.3. Static cutting method.

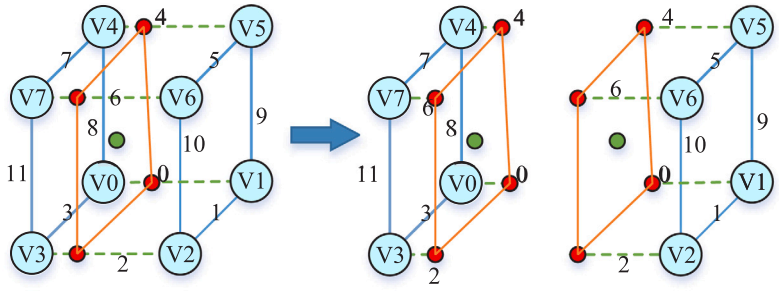

Fig. 2.4. Voxel connectivity graph disconnection.

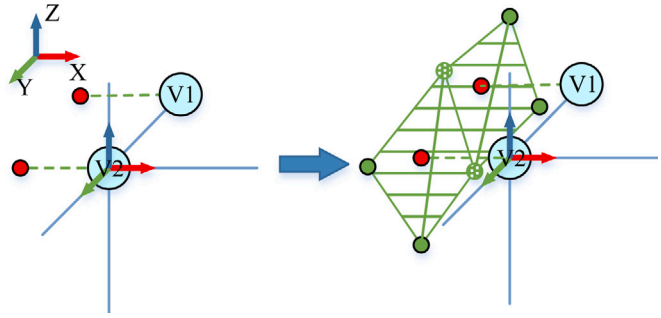

Fig. 2.5. Local surface rendering.

"0" indicates no intersection and "1" indicates intersection. When none of the edges intersect, the state is stored as the index "000000000000". When the *i*th edge is cut, the index can be quickly updated using a bitwise or operation to obtain the new connectivity.

Each vertex within a voxel has three edges. For each connected component, the edges that are cut and the cut points of all its vertices need to be identified. The feature point is the centroid of all the cut points. As shown in Fig. 2.4, the voxel is divided into two connected components, and each connected component has its corresponding feature point. Each edge that contains a vertex will participate in the calculation, and the cut edges will be calculated twice.

After all the voxel grids and edges are calculated, each particle's 6 edges and 8 neighboring quadrants are traversed. If an edge is cut, the corresponding feature points of the four neighboring quadrants of that edge are found for the particle. As shown in Fig. 2.5, V1 and V2 share two feature points, and they are connected respectively to form two triangles.

2.1.2. The dynamic cutting process

As shown in Fig. 2.6, in general, soft tissue undergoes deformation due to internal and external forces before cutting, causing its internal voxel grid to become distorted and no longer be a uniform hexahedron. The coordinate axes of the particles are rotated due to the influence of the surrounding connections. The calculation of the new coordinate system is based on the method described in [22].

Each coordinate axis' positive and negative directions are mapped to the six surrounding edges of the particle. When an edge is cut, the local coordinates of the cut point, relative to the two vertices of the edge, are recorded. This is done based on the condition that the edge has not undergone any deformation. As shown in the Fig. 2.7, when the cut is complete, the physical connection of that edge is severed, and the new cut point is computed using the corresponding coordinate axes.

2.1.3. Cutting path

In virtual surgery, real-time performance is a critical factor. Complex mesh structures increase computational burdens, especially during cutting operations, where a large amount of mesh data must be processed. This can lead to high computational costs and increased latency [32,33]. As shown in Fig. 2.8, to accurately detect the intersection between the cutting surgical instrument and the mesh during cutting, and to generate continuous cutting paths, this paper defines the surgical instrument as a cutting line. The cutting plane is formed by the cutting path of the previous and current frames, and the cutting plane consists of two triangles. This cutting plane persists in the simulation at any given frame. The advantage of this approach is that it prevents collision detection failure due to low frame rates or rapid movement of the surgical instrument. For collision detection between the cutting plane and the mesh, this paper employs the Möller-Trumbore algorithm for fast ray-triangle intersection tests. The core of this method is to determine intersection conditions by calculating the geometric relationship between the ray and the triangle, without

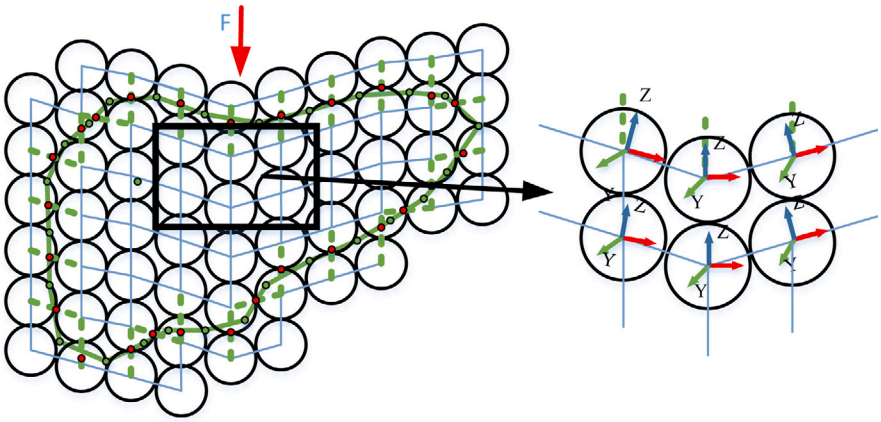

Fig. 2.6. The direction of the particles changes after deformation.

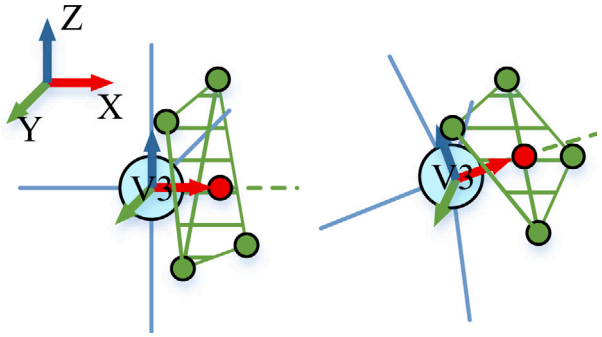

Fig. 2.7. Surface rendering after changing particle directions.

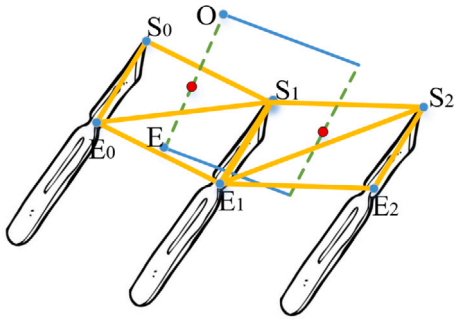

Fig. 2.8. Ray-based cutting method.

explicitly calculating the intersection point. This makes it more efficient than other algorithms that require solving quadratic equations, which is particularly important for large-scale parallel computation on hardware to enhance performance [34,35].

Let the ray origin and direction be $\boldsymbol{O}$ and $\boldsymbol{D}$, and the triangle vertices be $\boldsymbol{V}_0$, $\boldsymbol{V}_1$, and $\boldsymbol{V}_2$. The parameterized equation for the intersection point is:

$$\begin{cases} \boldsymbol{O} + t\boldsymbol{D} = \boldsymbol{V}_0 + u\boldsymbol{L}_1 + v\boldsymbol{L}_2 \\ \boldsymbol{L}_1 = \boldsymbol{V}_1 - \boldsymbol{V}_0 \\ \boldsymbol{L}_2 = \boldsymbol{V}_2 - \boldsymbol{V}_0 \end{cases} \tag{6}$$

The parameter t represents the ray parameter, and u and v are the barycentric coordinates that must satisfy the following conditions: $u \geq 0$, $v \geq 0$, and $u + v \leq 1$. The parameters are solved using Cramer's rule to solve the system of linear equations:

$$\begin{cases} \begin{bmatrix} \boldsymbol{D} & -\boldsymbol{L}_1 & -\boldsymbol{L}_2 \end{bmatrix} \begin{bmatrix} t \\ u \\ v \end{bmatrix} = \boldsymbol{T} \\ \boldsymbol{T} = \boldsymbol{V}_0 - \boldsymbol{O} \end{cases} \tag{7}$$

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{\begin{vmatrix} \boldsymbol{D} & -\boldsymbol{L}_1 & -\boldsymbol{L}_2 \end{vmatrix}} \begin{bmatrix} \begin{vmatrix} \boldsymbol{T} & -\boldsymbol{L}_1 & -\boldsymbol{L}_2 \end{vmatrix} \\ \begin{vmatrix} \boldsymbol{D} & \boldsymbol{T} & -\boldsymbol{L}_2 \end{vmatrix} \\ \begin{vmatrix} \boldsymbol{D} & -\boldsymbol{L}_1 & \boldsymbol{T} \end{vmatrix} \end{bmatrix} = \begin{bmatrix} (\boldsymbol{L}_2 \times \boldsymbol{T})\boldsymbol{L}_1 / ((\boldsymbol{L}_2 \times \boldsymbol{D})\boldsymbol{L}_1) \\ -(\boldsymbol{L}_2 \times \boldsymbol{D})\boldsymbol{T} / ((\boldsymbol{L}_2 \times \boldsymbol{D})\boldsymbol{L}_1) \\ (\boldsymbol{D} \times \boldsymbol{T})\boldsymbol{L}_1 / ((\boldsymbol{L}_2 \times \boldsymbol{D})\boldsymbol{L}_1) \end{bmatrix} \tag{8}$$

2.1.4. Ablation cutting process

In surgery, besides traditional blade cutting, electrocautery and ultrasound technologies are also widely used [36]. Electrocautery cutting is a technique where an electric current is directly applied to tissue, generating heat through high-frequency current to cut and coagulate tissue. Its advantages include reducing intraoperative bleeding, improving surgical precision, and enhancing safety. This type of cutting is often accompanied by some thermal damage and minimal volume loss. Similar volume loss simulations also exist in ophthalmic surgery, such as in cataract surgery, where ultrasound phacoemulsification is used to break the lens into small fragments and then aspirate them [37].

As shown in Fig. 2.9, the ablation cutting simulation process with volume loss is depicted. When the surgical instrument's cutting path passes through the soft tissue, the state of particles within a certain range is altered. Initially, the particles outside the isosurface are directly deleted and no longer participate in force calculations or surface generation. The particles initially within the isosurface are designated as "ablated particles", and the edges connecting these particles are marked as "ablated edges". At this point, the particles are deleted, cutting points are generated along the "ablated edges", and finally, all voxel grids are updated and the surface is redrawn.

2.2. Soft tissue modeling

Soft tissue modeling accurately describes the physical properties of the tissue, ensuring that the physical response of the cut tissue part is consistent with the uncutted part. The modeling needs to consider not only geometric deformations but also the internal mechanical behaviors, such as tension, compression, and bending. During the cutting process, the model helps calculate the forces exerted by the cutting surgical instrument on the soft tissue and how the tissue recovers or changes under the applied force.

In this paper, structural springs and bending springs based on hexahedral structured grids are applied to the computation of soft tissue. As

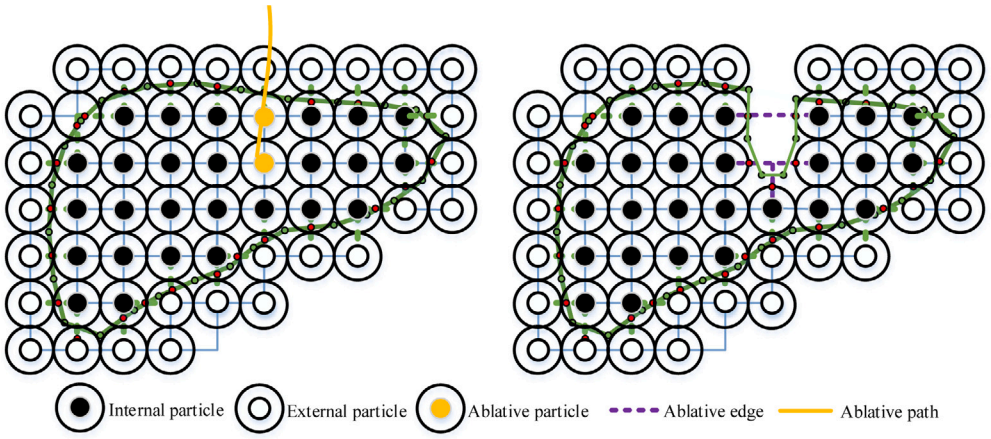

Fig. 2.9. Ablation cutting process.

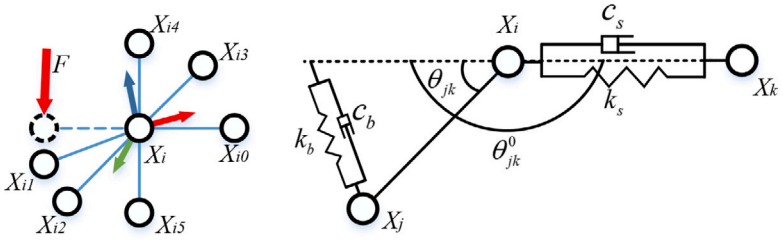

Fig. 2.10. Local elastic structure.

shown in Fig. 2.6, a two-dimensional schematic illustrates the deformation of the soft tissue grid. To ensure proper coupling between surface rendering and internal force calculation during soft tissue cutting, and to avoid scenarios where the soft tissue appears visually severed while forces are still acting on the separated ends, the physical model grid is used for surface rendering.s shown in Fig. 2.10, the Kelvin–Voigt model is adopted to simulate the viscoelasticity of soft tissue. Structural springs are used to model the stretching and compression of the soft tissue, while bending springs simulate resistance to bending and shear. The traditional diagonal shear springs in hexahedral grid methods are omitted in this approach.

For particle i, its motion equation is:

$$\boldsymbol{M}\ddot{\boldsymbol{x}}_i + \boldsymbol{F}_{\text{int}} = \boldsymbol{F}_{\text{ext}} \tag{9}$$

The internal force $\boldsymbol{F}_{\text{int}}$ consists of two components: the elastic force generated by linear stretching or compression $\boldsymbol{F^s}$, and the bending force induced by changes in the angle between adjacent springs $\boldsymbol{F^b}$:

$$\boldsymbol{F}_{\text{int}} = \boldsymbol{F}_i^s + \boldsymbol{F}_i^b \tag{10}$$

For any given particle, there can be up to six neighboring particles, six structural spring connections, and fifteen pairs of bending constraints. The force relationship between the central particle and any of its surrounding particles is given by:

$$\boldsymbol{F}_{ij}^s = -k_s \cdot \left\| \boldsymbol{x}_i - \boldsymbol{x}_j \right\| \cdot \frac{\boldsymbol{x}_i - \boldsymbol{x}_j}{\left\| \boldsymbol{x}_i - \boldsymbol{x}_j \right\|} - c_s \cdot (\dot{\boldsymbol{x}}_i - \dot{\boldsymbol{x}}_j) \tag{11}$$

where k_s and c_s represent the stiffness coefficient and damping coefficient of the structural spring, respectively. The total force exerted by the structural springs on particle i is:

$$\boldsymbol{F}_i^s = \sum_{j=0}^{5} \boldsymbol{F}_{ij}^s \tag{12}$$

For particle i and any two surrounding particles j and k forming the nth bending constraint, the bending moment it experiences is:

$$M_{in} = k_b \cdot (\theta_{jk} - \theta_{jk}^0) \tag{13}$$

The force experienced by particles j and k can be obtained by converting the bending moment as follows:

$$\begin{cases} \boldsymbol{F}_{ij}^{\text{M}} = \left(k_b \cdot (\theta_{jk} - \theta_{jk}^0) + c_b \dot{\theta} \right) \times (\boldsymbol{N}_{ik} \times \boldsymbol{N}_{ij}) \times \boldsymbol{N}_{ij} \\ \boldsymbol{F}_{ik}^{\text{M}} = \left(k_b \cdot (\theta_{jk} - \theta_{jk}^0) + c_b \dot{\theta} \right) \times (\boldsymbol{N}_{ij} \times \boldsymbol{N}_{ik}) \times \boldsymbol{N}_{ik} \end{cases} \tag{14}$$

where $\boldsymbol{N}_{ij} = \frac{\boldsymbol{x}_i - \boldsymbol{x}_j}{\left\| \boldsymbol{x}_i - \boldsymbol{x}_j \right\|}$ is the unit vector from particle i to particle j. The force on particle j under the influence of the bending moment can be expressed as:

$$\boldsymbol{F}_{in}^b = -(\boldsymbol{F}_{ij}^M + \boldsymbol{F}_{ik}^M) \tag{15}$$

The total force exerted on particle i by the bending springs can be expressed as:

$$\boldsymbol{F}_i^b = \sum_{n=0}^{14} \boldsymbol{F}_{in}^b \tag{16}$$

In soft tissue simulation, time integration methods are used to numerically solve the ordinary differential equations that describe the dynamic behavior of soft tissues. Choosing an appropriate time integration method is crucial for the stability and accuracy of the simulation results. Compared to implicit integration and small-step explicit integration, this paper adopts the adaptive step-size Runge–Kutta method, which can simultaneously meet the requirements for both solution accuracy and efficiency. With CUDA acceleration, this algorithm can be used to validate the cutting and force feedback algorithms presented in this paper. Given the current time step t_n, the position of the particle is $\boldsymbol{x_n}$, and the state vector of the particle is $\boldsymbol{y_n} = \left[\boldsymbol{x_n}\ \dot{\boldsymbol{x}}_n \right]^T$, the following can be obtained:

$$\boldsymbol{f}(t_n, \boldsymbol{y}_n) = \begin{bmatrix} \dot{\boldsymbol{x}}_n & \dfrac{\boldsymbol{F}_{\text{ext}} + \boldsymbol{F}_{\text{int}}}{m} \end{bmatrix}^T \tag{17}$$

For the time step h, the 7 slopes at $t_n = t_n + h$ can be calculated as follows in the Runge–Kutta method:

$$\begin{cases} k_1 = \boldsymbol{f}(t_n, \boldsymbol{y_n}) \\ k_2 = \boldsymbol{f}\left(t_n + \frac{1}{5}h, \boldsymbol{y_n} + h \cdot \left(\frac{1}{5}k_1\right)\right) \\ k_3 = \boldsymbol{f}\left(t_n + \frac{3}{10}h, \boldsymbol{y_n} + h \cdot \left(\frac{3}{40}k_1 + \frac{9}{40}k_2\right)\right) \\ k_4 = \boldsymbol{f}\left(t_n + \frac{4}{5}h, \boldsymbol{y_n} + h \cdot \left(\frac{44}{45}k_1 - \frac{56}{15}k_2 + \frac{32}{9}k_3\right)\right) \\ k_5 = \boldsymbol{f}\left(t_n + \frac{8}{9}h, \boldsymbol{y_n} + h \cdot \left(\frac{19372}{6561}k_1 - \frac{25360}{2187}k_2 + \frac{64448}{6561}k_3 - \frac{212}{729}k_4\right)\right) \\ k_6 = \boldsymbol{f}\left(t_n + h, \boldsymbol{y_n} + h \cdot \left(\frac{9017}{3168}k_1 - \frac{355}{33}k_2 + \frac{46732}{5247}k_3 + \frac{49}{176}k_4 \quad - \frac{5103}{18656}k_5\right)\right) \\ k_7 = \boldsymbol{f}\left(t_n + h, \boldsymbol{y_n} + h \cdot \left(\frac{35}{384}k_1 + \frac{500}{1113}k_3 + \frac{125}{192}k_4 - \frac{2187}{6784}k_5 + \frac{11}{84}k_6\right)\right) \end{cases} \tag{18}$$

The fourth-order solution and the fifth-order solution are respectively:

$$\begin{cases} \boldsymbol{y}_{n+1}^{(4)} = \boldsymbol{y}_n + \frac{5179}{57600}k_1 + \frac{7571}{16695}k_3 + \frac{393}{640}k_4 - \frac{92097}{339200}k_5 + \frac{187}{2100}k_6 + \frac{1}{40}k_7 \\ \boldsymbol{y}_{n+1}^{(5)} = \boldsymbol{y}_n + \frac{35}{384}k_1 + \frac{500}{1113}k_3 + \frac{125}{192}k_4 - \frac{2187}{6784}k_5 + \frac{11}{84}k_6 \end{cases} \tag{19}$$

The error can be obtained as:

$$\Delta = \left\| \boldsymbol{y}_{n+1}^{(5)} - \boldsymbol{y}_{n+1}^{(4)} \right\| \tag{20}$$

The new time step size h_{n} can be obtained based on the error as:

$$\begin{cases} \varepsilon = 10^{-3} \\ h_{\mathrm{n}} = h \cdot 0.9 \cdot (\frac{\Delta}{\varepsilon})^{1/5} \\ 0.2h < h_{\mathrm{n}} < 5h \end{cases} \tag{21}$$

where ε is the error threshold. The step size adjustment can theoretically generate any value, but unrestricted step size changes may lead to numerical issues or inefficiencies. Therefore, upper and lower bounds need to be set.

2.3. Haptic rendering

In order to provide realistic haptic feedback in virtual surgery, allowing users to clearly perceive the shape, stiffness, and other properties of the model's surface, appropriate force feedback calculation methods need to be established for both the surgical instrument and the organ model. Traditional force feedback methods mainly focus on physical modeling of the soft tissue itself, with the force often concentrated at the tip of the surgical instrument. This results in no force feedback when other parts of the surgical instrument collide, and even causes penetration issues [38,39]. A key problem in virtual surgery is how to establish collision constraints between the entire surgical instrument and the soft tissue [22,40].

This paper proposes an improvement based on the Finger-Proxy algorithm, simplifying the complex collision detection process between surgical instruments. As shown in Figs. 2.11 (b), (c), and (d), the surgical instrument is first approximated as a slender capsule, with its volume represented by a directed distance field. Unlike [22], the force feedback in this paper is no longer based on particle-surgical instrument collisions, but rather on collisions between the triangular faces of the soft tissue model and the soft tissue itself.

As shown in Fig. 2.11 (a), the Physical surgical instrument controls the position and orientation of the proxy surgical instrument. The proxy surgical instrument is displayed on the rendering interface, while the Physical surgical instrument is hidden. By calculating their position and orientation deviations in real-time, the proxy surgical instrument gradually approaches the Physical surgical instrument in discrete steps. Let the position of the Physical surgical instrument be denoted as $\boldsymbol{P}_A$,

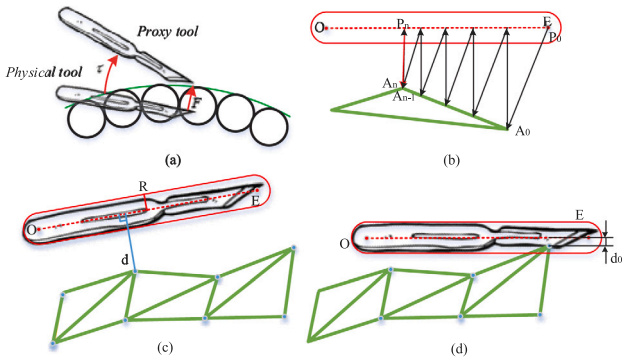

Fig. 2.11. Haptic model.(a)The proxy tool remains on the external surface, resulting in a positional and angular deviation from the actual tool. (b)The distance between the tool and the surface is iteratively calculated using the SDF of triangular faces. (c)The tool does not collide with the mesh. (d)The tool collides with the mesh.

the position of the proxy surgical instrument as $\boldsymbol{P}_B$, and the orientation quaternions as $\boldsymbol{q}_A$ and $\boldsymbol{q}_B$, respectively. The position deviation $\Delta\boldsymbol{P}$, the rotation axis $\boldsymbol{k}$, and the rotation angle θ can then be determined.

$$\Delta\boldsymbol{P} = \boldsymbol{P}_{\mathrm{B}} - \boldsymbol{P}_{\mathrm{A}} \tag{22}$$

$$\Delta\boldsymbol{q} = \boldsymbol{q}_B \otimes \boldsymbol{q}_A^{-1} = (\Delta\boldsymbol{q}_w, \Delta\boldsymbol{q}_x, \Delta\boldsymbol{q}_y, \Delta\boldsymbol{q}_z) \tag{23}$$

$$\theta = 2\arccos(\Delta\boldsymbol{q}_w) \tag{24}$$

$$\boldsymbol{k} = \frac{1}{\sin(\theta/2)} \begin{bmatrix} \Delta\boldsymbol{q}_x \\ \Delta\boldsymbol{q}_y \\ \Delta\boldsymbol{q}_z \end{bmatrix}, \quad \theta \neq 0 \tag{25}$$

As shown in Fig. 2.11 (b), the distance from the surgical instrument to each nearby triangle face is computed in parallel to determine the closest distance between the soft tissue and the surgical instrument. Both the surgical instrument and the triangular faces in space are convex shapes. When using the alternating projection method to compute the nearest point, each projection mapping is a non-expansive mapping. As a result, after each projection, the computed shortest distance is always less than or equal to that of the previous iteration.

$$\begin{cases} \mathbf{A}_n = \mathbf{SDF}(\mathrm{Triangle}, \mathbf{P}_n) \\ \mathbf{P}_{n+1} = \mathbf{SDF}(\mathrm{Tool}, \mathbf{A}_n) \\ \mathbf{D}_n = \|\mathbf{A}_n - \mathbf{P}_n\| \\ \mathbf{D}_{n+1} = \|\mathbf{P}_{n+1} - \mathbf{A}_n\| \\ \mathbf{D}_{n+1} \leq \mathbf{D}_n \end{cases} \tag{26}$$

where $\mathbf{A}_n$ denotes a point on the triangle, $\mathbf{P}_n$ denotes a point on the tool, $\mathbf{SDF}(\mathrm{Triangle}, \mathbf{P}_n)$ represents the mapping of $\mathbf{P}_n$ to its nearest point on the triangle, and $\mathbf{SDF}(\mathrm{Tool}, \mathbf{A}_n)$ represents the mapping of $\mathbf{A}_n$ to its nearest point on the tool.

As shown in Algorithm 1, an iterative optimization approach is used to obtain the closest distance and the nearest points of the two models. First, an arbitrary point $\boldsymbol{P_0}$ is selected on the surgical instrument as the starting point. Using the directed distance field function of the triangular face, the nearest point $\boldsymbol{A_0}$ and the direction are determined. Then, using the directed distance field of the surgical instrument, the nearest point on the surgical instrument is found, and the position of $\boldsymbol{P}$ is updated. When the magnitude of the two directed distances is smaller than the error threshold, stop the iteration. If penetration occurs, and the closest distance is smaller than the surgical instrument radius, the minimum separation direction is determined, and the proxy

Algorithm 1 Get the new position of Proxy tool after collision

Input: Triangles, Proxy Tool
Output: Proxy Tool Position ($\boldsymbol{P}_t$)
$\varepsilon \leftarrow 1 \times 10^{-3}$
$a \leftarrow 0.1$
Initialize Minimum Separation Vector $\boldsymbol{v}_{\text{ms}}$ to a large value
while True **do**
  for each triangle in Triangles **do**
    Select the starting point $\boldsymbol{P}_0$ on the tool
    $i \leftarrow 0$
    while $i < 100$ **do**
      $\boldsymbol{A}_0 \leftarrow$ SDF(triangle, $\boldsymbol{P}_0$)
      $\boldsymbol{P}_1 \leftarrow$ SDF(tool, $\boldsymbol{A}_0$)
      $\Delta \leftarrow \|\boldsymbol{P}_i - \boldsymbol{A}_i\| - \|\boldsymbol{P}_{i+1} - \boldsymbol{A}_i\|$
      if $|\Delta| < \varepsilon$ **then**
        $\boldsymbol{D} \leftarrow \boldsymbol{P}_i - \boldsymbol{A}_i$
        if $\|\boldsymbol{D}\| < \|\boldsymbol{v}_{\text{ms}}\|$ **then**
          $\boldsymbol{v}_{\text{ms}} \leftarrow \boldsymbol{D}$
          break
        end if
      end if
      $i \leftarrow i + 1$
    end while
  end for
  $\boldsymbol{P}_t \leftarrow \boldsymbol{P}_t + a \cdot \boldsymbol{v}_{\text{ms}}$
  if $\|\boldsymbol{v}_{\text{ms}}\| < \varepsilon$ **then**
    break
  end if
end while
Return the $\boldsymbol{P}_t$

surgical instrument is moved outward, ensuring that the proxy surgical instrument always stays on the surface of the soft tissue. At this point, the force feedback value is calculated based on the position and pose deviation of the two surgical instruments, and the reaction force is applied to the particles of the nearby triangle faces, as shown in Fig. 2.11 (a).

The force feedback calculation for the surgical instrument is as follows:

$$\begin{cases} \mathbf{F} = K_p \cdot \Delta\mathbf{P} - D_p \cdot \mathbf{v} \\ \boldsymbol{\tau} = K_o \cdot \theta \cdot \mathbf{k} - D_o \cdot \boldsymbol{\omega} \end{cases} \tag{27}$$

The force feedback applied to the surgical instrument consists of two components: a restoring term proportional to the position error and a damping term proportional to the velocity. The damping term only takes effect during contact to counteract oscillations. Where $\mathbf{F}$ is the linear force, K_p is the position stiffness coefficient, D_v is the linear velocity damping coefficient, $\mathbf{v}$ is the instrument's linear velocity, $\boldsymbol{\tau}$ is the torque, K_o is the orientation stiffness coefficient, D_o is the angular velocity damping coefficient, and $\boldsymbol{\omega}$ is the angular velocity vector.

3. Results

3.1. Experimental platform

In the virtual surgery platform presented in this paper, the server is configured with an NVIDIA RTX A2000 GPU and a 12th Gen Intel® Core™ i7-12700H processor. The software surgical instruments used include the Unity3D 2021.3 development engine, CUDA 12.5, Visual Studio 2022, and Matlab 2021. For the hardware part, the Omni haptic feedback device acts as both the force feedback device and the virtual surgical instrument manipulator. All computational processes in this paper are based on C++ and CUDA, with the rendering interface implemented using Unity3D and C#. The OpenHaptics plugin is used to connect Unity3D with the force feedback device, providing the ability to integrate 3D haptics into Unity and add haptic effects to the virtual world. The force feedback device converts the operator's control commands into mechanical motion data, which is input into the server to precisely control the movement of the surgical instrument in the virtual surgery environment. The server calculates and analyzes the input data, simulates the interaction between the surgical instrument and the soft tissue model, and then transmits the resulting geometric data and surgical instrument movement information to the display. Simultaneously, the server also transmits the force feedback from the model to the haptic feedback device, allowing the user to experience this feedback through the device's handle.

3.2. Virtual simulation computing framework

As shown in Fig. 3.1, the overall computational framework of this paper consists of two computational stages: the preprocessing stage and the real-time simulation stage, each sub-thread's output to other threads is highlighted with yellow boxes. In the preprocessing stage, the first step is to generate the background grid, based on which an outer surface is created by calculating the intersection points between the grid and the isosurface. The state information of the voxel corner points is then labeled. Afterward, the feature points within each voxel grid are computed using the GPU, and for each edge intersecting with the isosurface, feature points of the surrounding four voxels are connected to form two triangular faces, ultimately generating a continuous surface. In the real-time update stage, simulation calculations for the soft tissue are performed across three CPU threads. In the soft tissue modeling thread, the forces, positions, and orientations of particles are calculated, and the results are used to update the eight corner points of the background voxel grid in the geometry reconstruction thread. In the geometry reconstruction thread, the cutting operation is performed. On one hand, this affects the calculation of subsequent feature points and surface generation, and on the other hand, it changes the connectivity between soft tissue particles while updating their forces and orientations. After the surface is reconstructed, the triangular face information is used in the force feedback thread for collision detection between the surgical instrument and the soft tissue. In this thread, the proxy surgical instrument's position is updated in real-time to prevent penetration, and the position and orientation deviations between the proxy surgical instrument and the actual surgical instrument are calculated to compute the forces. Once the force feedback results are obtained, a reaction force threshold is set and applied to update the particle positions, ensuring smooth collision forces without penetration. The data communication rules between threads are that the same data should not be written to by two threads simultaneously on the CPU. If linear operations need to be performed on the data, atomic addition operations must be used in the GPU, and excessive atomic operations on the same thread are prohibited, otherwise, parallel computation would degrade to serial computation.

Our method is presented as a fully parallelized solution, featuring a multithreaded design framework that effectively avoids data races. It integrates soft tissue geometric modeling, physical modeling, and force feedback computation modules. The coupling between multiple threads in our approach does not affect the execution and output of individual threads. Even when the three threads run independently, users can ignore any visual or haptic discomfort caused by thread delays at a certain frame rate.

As shown in Fig. 3.2, the soft tissue calculation framework in this paper includes three sub-threads for data computation and one Unity visual rendering main thread. In the soft tissue simulation, the number of particles has the greatest impact on performance. The particles and their surrounding links are the most frequently used data in the GPU sub-threads. For different particle counts, the relationship between the frame rates of inter-thread operations is as follows:

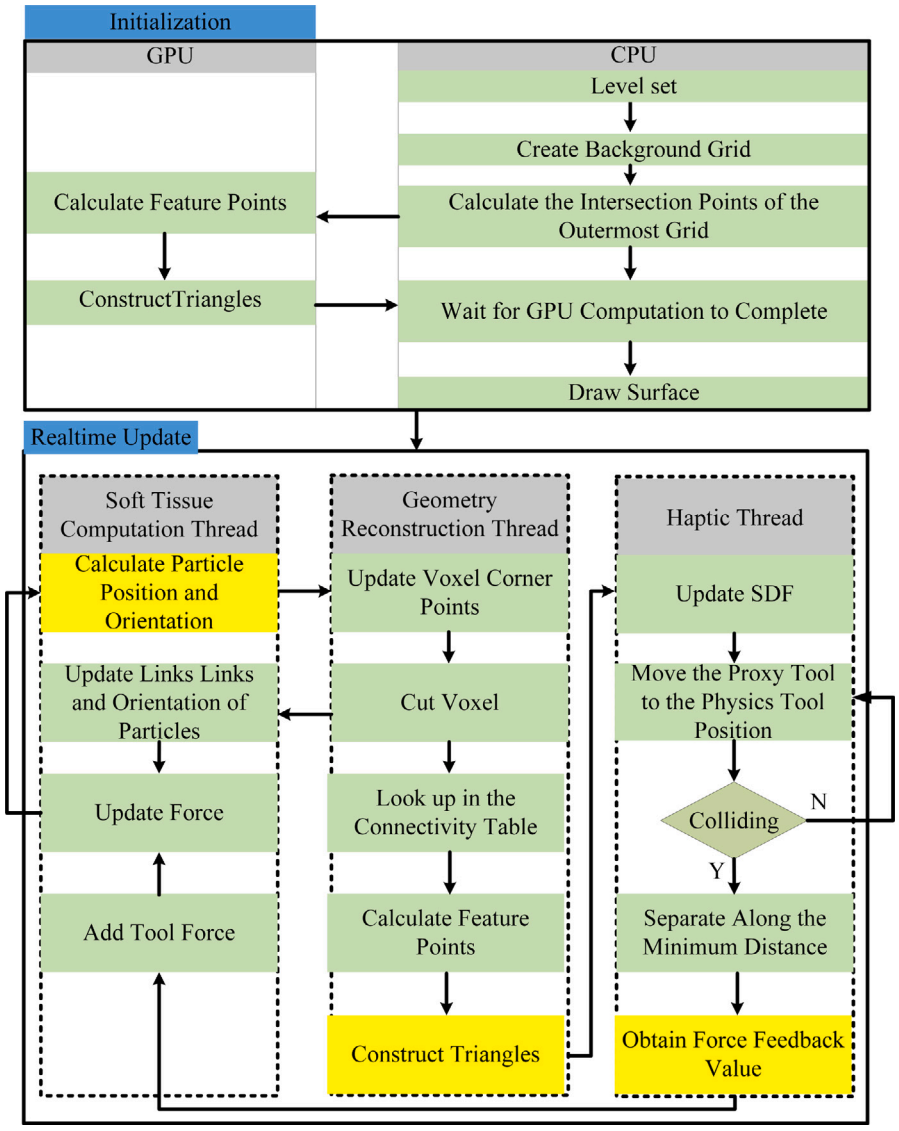

Fig. 3.1. Computing framework.

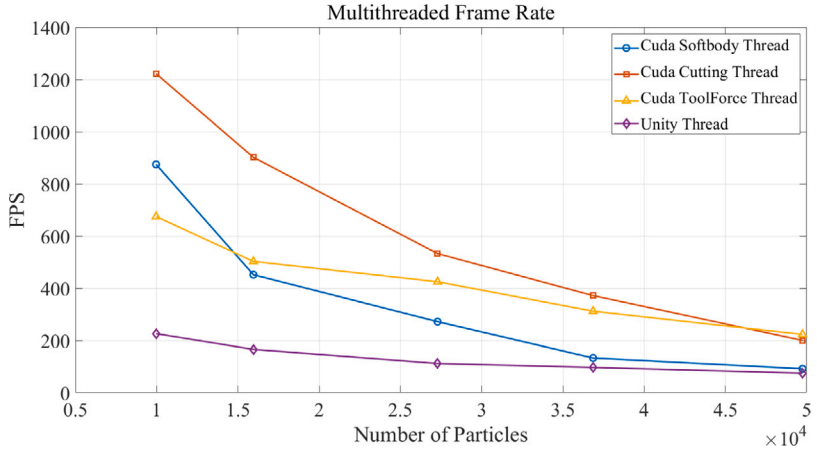

Fig. 3.2. The frame rates of different threads.

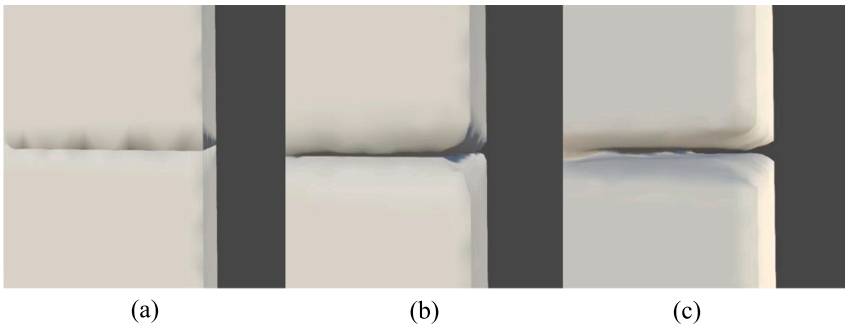

Fig. 3.3. Cutting results of the model using different Methods. (a) Tetrahedral Method. (b) Our Method. (c) MC Method.

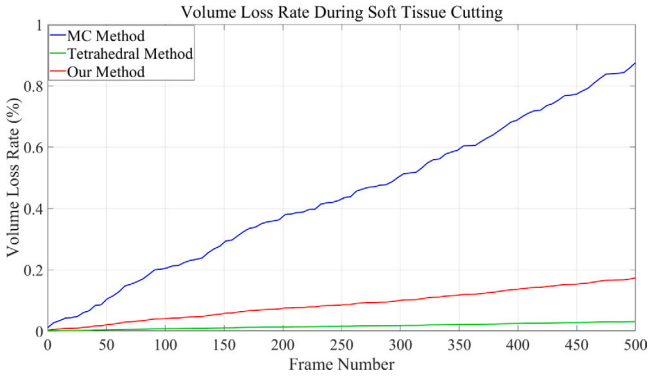

Fig. 3.4. Volume loss rate during soft tissue cutting.

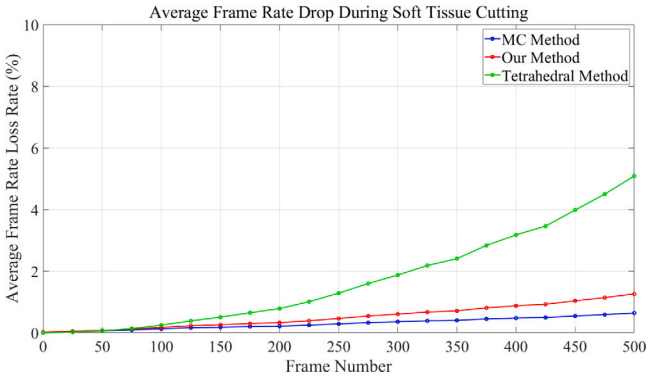

Fig. 3.5. Average frame rate drop during soft tissue cutting.

3.3. Comparison

This paper experimentally verifies the advantages of the proposed cutting algorithm by comparing it with the traditional Marching Cubes (MC) cutting algorithm and the tetrahedral cutting algorithm. Compared to the MC method, the proposed approach ensures minimal volume loss during cutting while retaining the ablation effect of the MC algorithm. This allows the simulation to maintain small incision gaps when using a blade while also adapting to surgical procedures involving ablation effects, such as electrocautery or ultrasonic emulsification. Compared to traditional tetrahedral cutting, the proposed method avoids dynamically adding new particles or voxels and eliminates concerns about the impact of distorted triangular faces or tetrahedra on mechanical simulation. As a result, it prevents the increase in computation time caused by the growing number of cutting surfaces.

As shown in Figs. 3.3 and 3.4, under zero-gravity conditions, the volume loss rate of soft tissue gradually increases as cutting progresses; if this trend continues, the tissue volume will eventually approach zero after multiple cuts. This paper compares the volume loss rates during cutting among the MC cutting method, the tetrahedral cutting method, and the proposed method. Since the tetrahedral method cuts along the mesh edges, volume loss mainly arises from slight smoothing of vertices, resulting in relatively less volume loss. However, both the MC method and the proposed DC method use an approximate solution based on vertex averaging when processing internal vertices, causing more noticeable volume loss at the edges. Compared to the MC method, the proposed method maintains a relatively lower loss rate on internal cut surfaces, demonstrating the ability to handle progressive cutting and preserving a certain degree of realism during small-scale cuts.

As shown in Fig. 3.5, a comparison of frame rates during the cutting process between the proposed method and other methods is presented. During continuous cutting, we estimate the approximate frames processed per second by measuring the execution time of each frame. Every 50 frames, we calculate the average frame rate over the preceding 50 frames and compare it to the frame rate before cutting to determine the average frame rate loss.

The figure clearly demonstrates that, compared to the tetrahedral cutting method, the proposed method maintains a relatively stable frame rate during cutting, with a slower rate of decline. In the tetrahedral cutting method, tetrahedral voxels are subdivided to generate smaller voxels, helping to ensure volume stability during blade cutting. However, as more voxels are generated during the cutting process, mechanical relationships must be re-established, and the impact of elongated triangles on mechanical simulations needs to be considered. This increases computational complexity, affecting the efficiency of soft tissue cutting. In contrast, the proposed method does not introduce new particles but only increases the number of voxels that need to be rendered, optimizing the speed of frame rate decline and bringing its performance loss rate closer to that of the MC algorithm.

When handling mesh splitting, compared with the classical approach of soft tissue modeling using complete hexahedral meshes combined with the dual-contouring method, our approach directly computes the positions of feature points using the coordinate frames of six-degree-of-freedom particles, thereby avoiding the need to update the full mesh vertex list. Since each particle's update is influenced by surrounding particles, updating the coordinate frame of each particle introduces additional computation; however, this operation can be efficiently parallelized, with each thread performing at most six atomic additions per particle. Timing comparisons of the mesh update process indicate that our method achieves a clear advantage in mesh reconstruction speed, as shown in Table 1.

As shown in Fig. 3.6, in traditional soft tissue simulations, the interaction between the surgical tool and soft tissue is typically modeled using the Finger-Proxy algorithm within the Chai3D framework for force feedback, or by directly applying force to individual particles or groups of particles. However, this method does not ensure that parts of

Table 1
Mesh reconstruction time for different particle counts.

Time (ms) / Particle Count / Method	9580	11103	12626	18617	24609	29842	35075	44688	54301
Hexahedral+DC	5.13	5.70	6.27	9.17	12.07	13.91	15.75	19.84	23.92
Ours	0.87	1.00	1.12	1.52	1.91	2.21	2.51	4.08	5.66

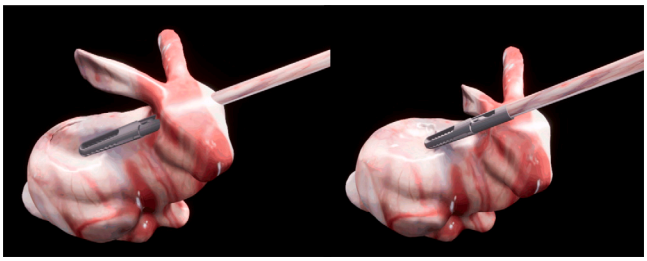

Fig. 3.6. Comparison of two force feedback models. Left: the classic point-to-surface(Finger-Proxy) collision model. Right: the improved volume-to-surface collision model.

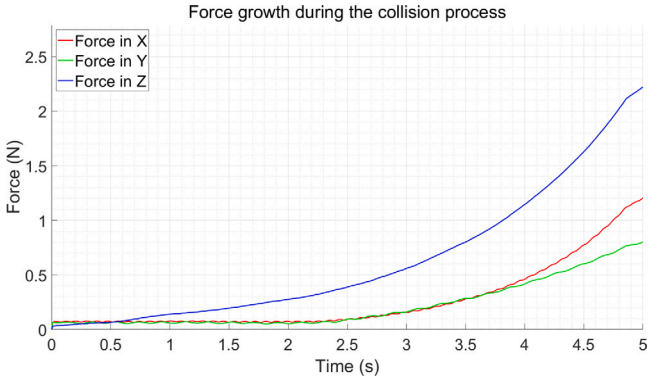

Fig. 3.7. Force growth during the collision process.

the surgical tool other than the tip have a collision volume, which can result in model penetration, as illustrated in the left image. When an approximate tool presses against particles, if the applied pressure exceeds a certain threshold, the model may experience excessive deformation, or the tool may penetrate the tissue. To address this issue, the proposed method uses a signed distance field to approximate the representation of the surgical tool. By combining this with the force rendering concept of the Finger-Proxy model, it ensures that the surgical tool remains outside the soft tissue, thereby preventing the penetration problem, as shown in the right image. As shown in Fig. 3.7, the force feedback curve after smoothing during the collision process is presented. When the tool presses down from top to bottom, the force along the z-axis gradually increases at the initial moment. After approximately 2 s, the tool collides at other positions, and at this point, the forces along the x and y axes also begin to gradually increase.

3.4. Interaction simulation

To further validate the practical effectiveness of our algorithm in applications, simulation experiments on soft tissue interactions under different conditions were conducted. These experiments mainly included soft tissue cutting, tearing, and collision effects.

In our model, the voxel partitioning is based on the AABB bounding box, a total of 54,301 particles were used for soft tissue surface rendering and dynamic modeling, the voxel size was set to 0.25 cm. The stiffness coefficient k_s was set to 7.5×10^4 N/m, the bending coefficient k_b to 2×10^4 N/m, with damping coefficients $c_s = 0.92$ and $c_b = 0.9$. The time step was set to 0.02 s.

The results show that the proposed method performs excellently in soft tissue cutting and collision interactions. As shown in Fig. 3.8, the cutting results under zero gravity conditions. The algorithm proposed in this paper supports cutting with varying widths. By changing the thickness D of the surgical instrument, two cutting points are generated on the cutting edge and bound to the particles at both ends. The figure illustrates the process of gradually increasing the cutting width from zero. It can be seen from the figure that as the blade width increases, the cutting gap gradually widens. Fig. 3.9 shows the cutting results when a point of the soft tissue is fixed and it naturally falls under the influence of gravity. During the cutting operation, the soft tissue below the cut gradually opens due to gravity. From the dashed line in the figure, it can be seen that the tissue in the upper half moves upward by a certain distance before and after the cut, successfully disconnecting the physical connection between the upper and lower halves, thereby reducing the overall gravity of the soft tissue. After the particle position changes, the cutting point can adjust its world coordinates according to the position and orientation of the corresponding particle. In the fourth part of the figure, it can be observed that even when the particles undergo significant displacement and the soft tissue posture changes noticeably, the mesh still displays correctly.

Fig. 3.10 shows the cut surfaces of soft tissue after multiple cuts, including cross cuts, multiple planar cuts, and curved cuts. After cutting and deformation under applied forces, the method is able to maintain the integrity of the cut surfaces and the continuity of the textures, ensuring visual realism. It supports various cutting conditions and can adapt to surgical scenarios requiring multiple cuts. Fig. 3.11 illustrates the results of the collision between soft tissue and surgical instruments. By computing the minimum distance between the surgical instrument and the soft tissue using the proposed method, the position of the surgical instrument can be updated in real time. An elastic connection is established between the proxy surgical instrument and the physical surgical instrument to ensure that the instrument remains on the surface of the soft tissue while exerting forces on both the tissue and the user. As shown in the figure, even under large deformations, the surgical tool and the soft tissue can maintain the correct relative geometric relationship.

Fig. 3.12 illustrates the process of dragging the soft tissue after cutting, with cutting continuing during the dragging operation. As shown in the figure, the cutting surface fitted by the proposed method accurately follows the direction of the applied force, making it suitable for simulating scenarios in surgical environments where incisions are further cut after being stretched.

Fig. 3.13 present the cutting results of different soft tissues and the results of ablation cutting. This method can be applied to liver cutting, vascular or intestinal cutting, as well as surgical simulations such as phacoemulsification and lens fragmentation in cataract treatment. The proposed approach demonstrates good adaptability to surfaces of varying complexity.

4. Discussion

In soft tissue modeling and simulation, surface mesh processing methods typically include MC, DC, and the tetrahedral method [21–23,26]. These methods are designed to utilize mesh nodes for surface rendering and dynamic simulation. The MC and DC methods generally employ structured meshes for mechanical simulation, whereas the tetrahedral method directly uses the vertices of triangular meshes as nodes for dynamic simulation. To address the volume loss and frame

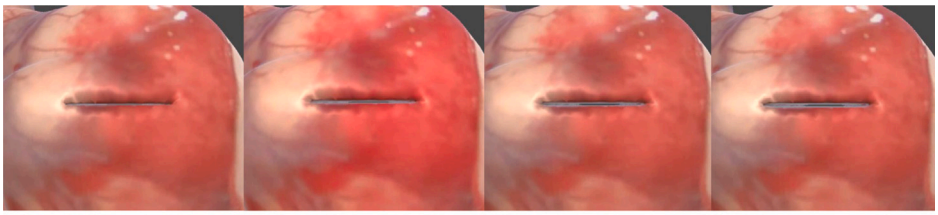

Fig. 3.8. The result of changing the cutting width under zero gravity.

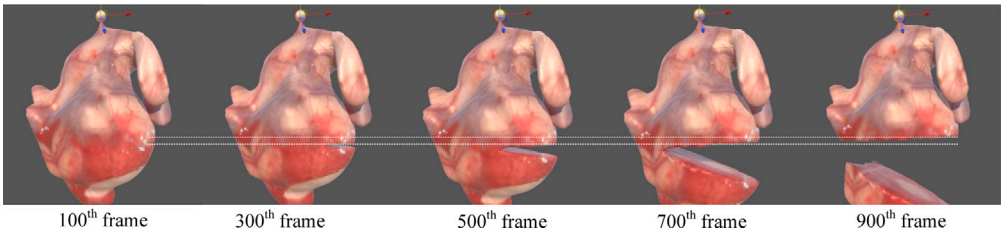

Fig. 3.9. The cutting result under the influence of gravity with a fixed point.

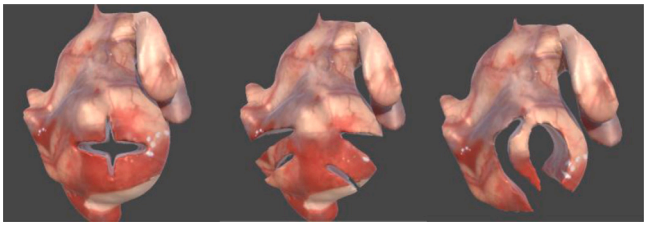

Fig. 3.10. Soft tissue after multiple cuts.

rate degradation caused by an increasing number of cutting planes in soft tissue cutting simulation, this paper proposes a novel parallelized DC strategy that adapts to dynamic and fractured mesh rendering. We preprocess all possible cutting cases of hexahedral voxel grids and encapsulate them into a lookup table. The corresponding lookup indices are stored in binary form, allowing for quick retrieval of mesh connectivity when computing feature points. This enables the determination of the number of feature points within each voxel and the contribution value of each edge to these feature points, as illustrated in Fig. 3.2. Our method maintains real-time performance even when simulating approximately 50,000 particles. Instead of duplicating or subdividing the mesh, our approach processes the cut mesh directly, preventing additional computational overhead in dynamic simulation. As shown in Fig. 3.4, during continuous cutting, the decrease in frame rate is relatively slow. In contrast, when using the tetrahedral subdivision algorithm for cutting, the frame rate declines more rapidly. Our cutting method balances low-volume loss cutting and real-time performance. However, our approach does not yet consider how to handle fractured meshes after adaptive mesh cutting. When using mechanical simulation meshes for 3D reconstruction, even if mechanical modeling and geometric modeling run on separate threads, the computational load of the mechanical simulation thread must be considered. Therefore, the number of particles is limited. On GPU computing, our method extensively utilizes atomic operations. Specifically, when multiple springs exert forces on the same particle, force accumulation requires atomic operations. The next computational step must wait until all GPU threads complete this force accumulation. Additionally, mesh smoothing also relies on atomic operations. Since multiple triangular faces share the same vertex, and each triangle contributes to the normal vector calculation in different GPU threads, the final normal computation must wait for all threads to finish. Otherwise, rendering artifacts such as flickering or blackened surfaces may occur. Future work will focus on designing algorithms that fully exploit GPU hardware resources. Beyond model processing, we also consider force haptic rendering in human–computer interaction. When handling collisions between tools and soft tissue, we refer to the methods proposed by Zhang, Berndt, et al. [22,40], which approximate the shape of surgical instruments using bounding boxes. Combining the Finger-Proxy algorithm [38] and signed distance fields, we ensure that surgical tools remain on the outer surface of the soft tissue. This prevents penetration even when excessive speed or force is applied during operation.

5. Conclusion

This paper proposes a virtual simulation algorithm applicable to both blade cutting and blunt surgical instrument cutting, providing a novel solution for force feedback between soft tissue and surgical instruments. To meet real-time requirements, the algorithm's components are parallelized using the CUDA framework for accelerated computation, and the complexity of the geometric modeling algorithm is optimized. Cutting simulation experiments under various working conditions were conducted to validate the effectiveness of both blade cutting and blunt surgical instrument ablation cutting.

This study still has certain limitations. For example, the integration of physical modeling and geometric modeling within adaptive meshing has not yet been achieved, there is still significant volume loss, and unlimited cutting is not yet supported, which poses challenges for simulating thin-walled heterogeneous soft tissues. Compared with traditional mesh cutting methods, due to feature point drift, our approach may cause a sudden separation of the two sides of the cutting surface over a large distance, rather than a smooth split, under large deformations. To address this issue, we interpolate the positions of post-cut feature points from the previous frame to the current frame, thereby achieving visually smooth transitions. However, this strategy does not fully adhere to physical principles and requires further refinement.

In future work, we will continue to explore techniques in virtual surgery simulation, with particular focus on biomechanical modeling algorithms, aiming to improve the mechanical performance and interactive experience of virtual soft tissues. The video related to the work presented in this paper can be found at the following link: https://youtu.be/eK_QKEXwyag.

Fig. 3.11. Collision and grasping of soft tissues in different regions.

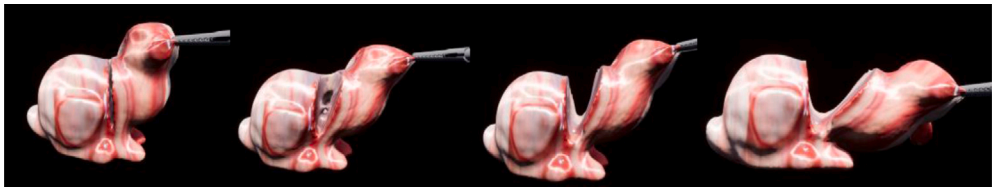

Fig. 3.12. Soft tissue being pulled while being cut.

Table 2
Explanation of key terms in Fig. 2.2.

Term	Description
Voxel grid	A volumetric grid used for 3D reconstruction.
Intersecting edge	An edge that intersects with the isosurface.
Level Set	A point in the volumetric data where the level set function equals zero, typically representing an implicitly defined surface.
Intersecting Point	The point where an edge intersects with the isosurface or the cutting plane.
Internal Corner Point	A point located inside the isosurface.
External Corner Point	A point located outside the isosurface.
Initial Average Point	The initial feature point obtained by averaging intersecting points.
Feature Point	A refined point after iterative update from the initial average point.
Normal	The surface normal at the intersecting point.
Direction	The update direction of the feature point.

Table 3
Explanation of key terms in Fig. 2.3.

Term	Description
Cutting edge	An edge that intersects with the cutting plane.
Cutting line	The trajectory of the cutting path.
Intersecting point	The point where an edge intersects with the isosurface or the cutting plane.
External feature point	A feature point located on the outer surface.
Internal feature point	A feature point located on the inner surface.

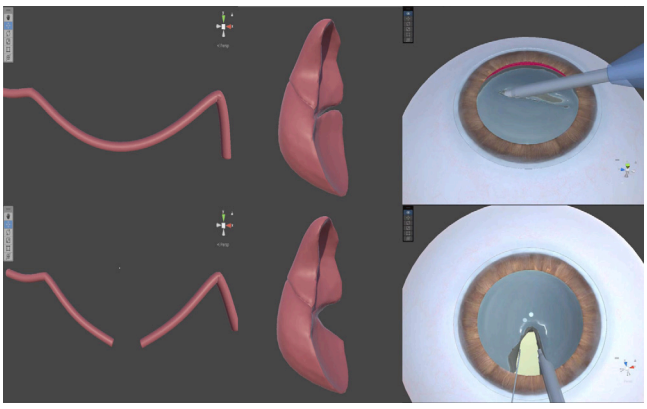

Fig. 3.13. Cutting results of different soft tissues.

CRediT authorship contribution statement

Liang Li: Writing – original draft, Methodology, Investigation, Data curation, Conceptualization. **Lai Zhou:** Writing – review & editing, Methodology, Funding acquisition. **Xueyu Zhou:** Software, Investigation.

Ethics statement

The author certify that this manuscript is original and has not been published and will not be submitted elsewhere for publication while being considered by Computer Methods and Programs in Biomedicine. And the study is not split up into several parts to increase the quantity of submissions and submitted to various journals or to one journal over time. No data have been fabricated or manipulated (including images) to support your conclusions. No data, text, or theories by others are presented as if they were our own. The submission has been received explicitly from all co-authors. And authors whose names appear on the submission have contributed sufficiently to the scientific work and therefore share collective responsibility and accountability for the results.

Declaration of competing interest

The authors declare that they do not have any commercial or associative interest that represents a conflict of interest in connection with the work submitted.

Acknowledgments

This work was supported by the 2022 Shanghai Rising Star Program (including Sailing Program) [grant numbers No. 22YF1417400].

Appendix. Nomenclature tables

See Tables 2 and 3

References

[1] A.R. See, J.A.G. Choco, K. Chandramohan, Touch, texture and haptic feedback: a review on how we feel the world around us, Appl. Sci. 12 (9) (2022) 4686, http://dx.doi.org/10.3390/app12094686.
[2] K. McCloskey, R. Turlip, H.S. Ahmad, Y.G. Ghenbot, D. Chauhan, J.W. Yoon, Virtual and augmented reality in spine surgery: a systematic review, World Neurosurg. 173 (2023) 96–107, http://dx.doi.org/10.1016/j.wneu.2023.02.068.
[3] D. Zuo, M. Zhu, D. Chen, Q. Xue, S. Avril, K. Hackl, Y. He, Three-dimensional anisotropic unified continuum model for simulating the healing of damaged soft biological tissues, Biomech. Model. Mechanobiol. 23 (6) (2024) 2193–2212, http://dx.doi.org/10.1007/s10237-024-01888-6.
[4] P. Madhikar, J. Åström, M. Karttunen, Cellsim3d: Gpu accelerated software for simulations of cellular growth and division in three dimensions, Comput. Phys. Comm. 232 (2018) 206–213, http://dx.doi.org/10.1016/j.cpc.2018.05.024.
[5] A. Ballit, T.-T. Dao, Hypermsm: A new msm variant for efficient simulation of dynamic soft-tissue deformations, Comput. Methods Programs Biomed. 216 (2022) 106659, http://dx.doi.org/10.1016/j.cmpb.2022.106659.
[6] H. Xie, J. Song, B. Gao, Y. Zhong, C. Gu, K.-S. Choi, Finite-element Kalman filter with state constraint for dynamic soft tissue modelling, Comput. Biol. Med. 135 (2021) 104594, http://dx.doi.org/10.1016/j.compbiomed.2021.104594.
[7] M. Macklin, M. Müller, N. Chentanez, Xpbd: position-based simulation of compliant constrained dynamics, in: Proceedings of the 9th International Conference on Motion in Games, 2016, pp. 49–54, http://dx.doi.org/10.1145/2994258.2994272.
[8] S.F. Johnsen, Z.A. Taylor, M.J. Clarkson, J. Hipwell, M. Modat, B. Eiben, L. Han, Y. Hu, T. Mertzanidou, D.J. Hawkes, et al., Niftysim: A gpu-based nonlinear finite element package for simulation of soft tissue biomechanics, Int. J. Comput. Assist. Radiol. Surg. 10 (2015) 1077–1095, http://dx.doi.org/10.1007/s11548-014-1118-5.
[9] S. Hang, Tetgen, a delaunay-based quality tetrahedral mesh generator, ACM Trans. Math. Software 41 (2) (2015) 11, http://dx.doi.org/10.1145/2629697.
[10] M. Cardoso, B. Feijóo, A.P. Castro, F. Ribeiro, P.R. Fernandes, Axisymmetric finite element modelling of the human lens complex under cataract surgery, Symmetry 13 (4) (2021) 696, http://dx.doi.org/10.3390/sym13040696.
[11] D. Marinković, M. Zehn, Corotational finite element formulation for virtual-reality based surgery simulators, Phys. Mesomech. 21 (2018) 15–23, http://dx.doi.org/10.1134/S1029959918010034.
[12] H.P. Bui, S. Tomar, S.P. Bordas, Corotational cut finite element method for real-time surgical simulation: Application to needle insertion simulation, Comput. Methods Appl. Mech. Engrg. 345 (2019) 183–211, http://dx.doi.org/10.1016/j.cma.2018.10.023.
[13] H. Xie, J. Song, Y. Zhong, C. Gu, K.-S. Choi, Constrained finite element method for runtime modeling of soft tissue deformation, Appl. Math. Model. 109 (2022) 599–612, http://dx.doi.org/10.1016/j.apm.2022.05.020.
[14] W. Xu, Y. Wang, W. Huang, Y. Duan, An efficient nonlinear mass–spring model for anatomical virtual reality, IEEE Trans. Instrum. Meas. 71 (2022) 1–10, http://dx.doi.org/10.1109/TIM.2022.3164132.
[15] Y. Duan, W. Huang, H. Chang, W. Chen, J. Zhou, S.K. Teo, Y. Su, C.K. Chui, S. Chang, Volume preserved mass–spring model with novel constraints for soft tissue deformation, IEEE J. Biomed. Heal. Inform. 20 (1) (2014) 268–280, http://dx.doi.org/10.1109/JBHI.2014.2370059.
[16] H. Va, M.-H. Choi, M. Hong, Real-time surface-based volume constraints on mass–spring model in unity3d, IEEE Access 11 (2023) 17857–17869, http://dx.doi.org/10.1109/ACCESS.2023.3245130.
[17] J. Zhang, Y. Zhong, C. Gu, Soft tissue deformation modelling through neural dynamics-based reaction–diffusion mechanics, Med. Biol. Eng. Comput. 56 (2018) 2163–2176, http://dx.doi.org/10.1007/s11517-018-1849-5.
[18] X. Zhang, Z. Wang, W. Sun, S. Jha, et al., Heterogeneous soft tissue deformation model based on cellular neural networks: Application in pulmonary hamartomas surgery, Biomed. Signal Process. Control. 95 (2024) 106290, http://dx.doi.org/10.1016/j.bspc.2024.106290.
[19] J. Li, T. Liu, L. Kavan, B. Chen, Interactive cutting and tearing in projective dynamics with progressive cholesky updates, ACM Trans. Graph. 40 (6) (2021) 1–12, http://dx.doi.org/10.1145/3478513.3480505.
[20] W. Hongyu, Y. Haonan, Y. Fan, S. Jian, G. Yuan, T. Ke, H. Aimin, Interactive hepatic parenchymal transection simulation with haptic feedback, Virtual Real. Intell. Hardw. 3 (5) (2021) 383–396, http://dx.doi.org/10.1016/j.vrih.2021.09.003.
[21] G. Jeřábková, S. Barbier, F. Faure, J. Allard, Volumetric modeling and interactive cutting of deformable bodies, Prog. Biophys. Mol. Biol. 103 (2–3) (2010) 217–224, http://dx.doi.org/10.1016/j.pbiomolbio.2010.09.012.
[22] I. Berndt, R. Torchelsen, A. Maciel, Efficient surgical cutting with position-based dynamics, IEEE Comput. Graph. Appl. 37 (3) (2017) 24–31, http://dx.doi.org/10.1109/MCG.2017.45.
[23] P. Yu, Z. Zhao, R. Wang, J. Pan, Real-time soft body dissection simulation with parallelized graph-based shape matching on gpu, Comput. Methods Programs Biomed. 250 (2024) 108171, http://dx.doi.org/10.1016/j.cmpb.2024.108171.
[24] J. Wu, C. Dick, R. Westermann, Interactive high-resolution boundary surfaces for deformable bodies with changing topology, Vriphys 11 (2011) 29–38.
[25] J. Wu, R. Westermann, C. Dick, Real-time haptic cutting of high-resolution soft tissues, 2014, pp. 469–475.
[26] S. Chen, R. Yang, L. Ma, M. Xu, Y. Zhou, Hybrid optimization-based cutting simulation for soft objects, Comput.-Aided Des. 163 (2023) 103561, http://dx.doi.org/10.1016/j.cad.2023.103561.
[27] W. Shi, P.X. Liu, M. Zheng, Cutting procedures with improved visual effects and haptic interaction for surgical simulation systems, Comput. Methods Programs Biomed. 184 (2020) 105270.
[28] W. Shi, P.X. Liu, M. Zheng, A new volumetric geometric model for cutting procedures in surgical simulation, Comput. Methods Programs Biomed. 178 (2019) 77–84.
[29] Y. Tang, S. Liu, Y. Deng, Y. Zhang, L. Yin, W. Zheng, An improved method for soft tissue modeling, Biomed. Signal Process. Control. 65 (2021) 102367, http://dx.doi.org/10.1016/j.bspc.2020.102367.
[30] X. Zhang, W. Zhang, W. Sun, A. Song, A new soft tissue deformation model based on Runge-Kutta: application in lung, Comput. Biol. Med. 148 (2022) 105811, http://dx.doi.org/10.1016/j.compbiomed.2022.105811.
[31] L.A. Schmitz, Analysis and acceleration of high quality isosurface contouring, 2009, http://hdl.handle.net/10183/151064.
[32] C.R. Koger, S.S. Hassan, J. Yuan, Y. Ding, Virtual reality for interactive medical analysis, Front. Virtual Real. 3 (2022) 782854, http://dx.doi.org/10.3389/frvir.2022.782854.
[33] J. Zhang, J. Qian, H. Zhang, L. He, B. Li, J. Qin, H. Dai, W. Tang, W. Tian, Maxillofacial surgical simulation system with haptic feedback, J. Ind. Manag. Optim. 17 (6) (2021) http://dx.doi.org/10.3934/jimo.2020137.
[34] T. Mller, B. Trumbore, Fast, Minimum Storage Ray-Triangle Intersection, ACM, 2005, http://dx.doi.org/10.1145/1198555.1198746.
[35] V. Shumskiy, Gpu ray tracing–comparative study on ray-triangle intersection algorithms, 2013, pp. 78–91, http://dx.doi.org/10.1007/978-3-642-39759-2_6.
[36] S. Samal, R. Ramchandani, D. Mohanty, Comparison of intraoperative and postoperative outcomes with skin flaps raised using either harmonic scalpels or electrocautery during modified radical mastectomy, Cureus 16 (6) (2024) http://dx.doi.org/10.7759/cureus.62320.
[37] S.C. Ugurbas, S. Caliskan, A. Alpay, S.H. Ugurbas, Impact of intelligent phacoemulsification software on torsional phacoemulsification surgery, Clin. Ophthalmol. (2012) 1493–1498, http://dx.doi.org/10.2147/OPTH.S35283.
[38] Y. Liu, B. Liu, A review of virtual surgical object modeling technology based on force feedback, Int. J. Adv. Netw. Monit. Control. 7 (2) (2022) 24–31, http://dx.doi.org/10.2478/ijanmc-2022-0013.
[39] Y. Liu, B. Liu, A review of virtual surgical object modeling technology based on force feedback, in: Monitoring and Control (ANMC) Cooperate, vol. 24, Xi'an Technological University (CHINA) West Virginia University (USA) Huddersfield University of UK (UK), 2022, http://dx.doi.org/10.2478/ijanmc-2022-0013.
[40] X. Zhang, J. Duan, W. Sun, T. Xu, S.K. Jha, A three-stage cutting simulation system based on mass–spring model, Comput. Model. Eng. Sci. 127 (1) (2021) 117–133, http://dx.doi.org/10.32604/cmes.2021.012034.